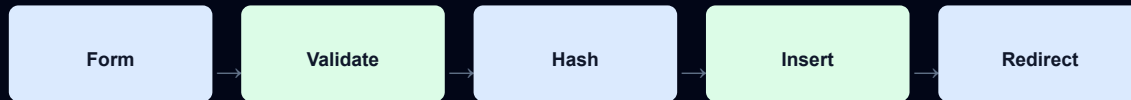


Registration Module

Student Notes System / Student Notes Manager

Today we convert a static registration page into a real database-backed feature.



Today's milestone

Validated users stored in MySQL using a safe registration flow.

Security mindset

Browser input is untrusted. Validate before database work.

Trainer goal

Students must explain each layer: browser, Flask, validation, database, response.

Trainer

Madhu Parvathaneni, CTO - Codegnan
madhu@codegnan.com - 8121289993

Today's learning map

Students should leave with a working registration flow and a clear mental model.



Form behavior

GET displays the page; POST submits form data.

request.form

Read browser-submitted name-value pairs safely.

Validation

Reject missing, malformed or weak inputs before SQL.

MySQL connection

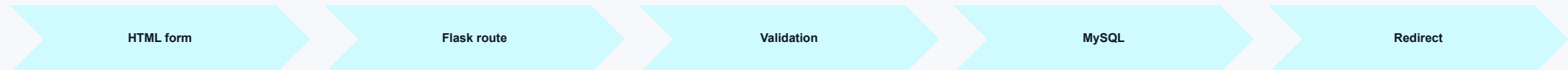
Use connector, cursor, execute, commit and fetchone.

Parameterized SQL

Bind values separately; never concatenate user input.

Registration flow

Duplicate check, password hash, insert and redirect.



Quick recall from Day 8

Registration needs yesterday's template skills.



```
1 {% extends 'base.html' %}
2
3 {% block content %}
4     <form method="POST">
5         <input name="email">
6     </form>
7 {% endblock %}
```

Template inheritance

base.html gives the common layout; register.html fills the content block.

Static boilerplate

CSS is provided; students focus on Flask, Jinja and form handling.

Form fields

Flask can only read fields that have a name attribute.

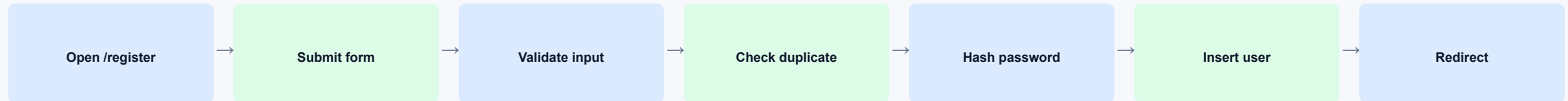
Today's upgrade

The form now creates a real row inside users table.

Ask: Which value will Flask receive from `<input name="email">`? What happens if name is missing?

Day 9 target milestone

From a registration screen to a database-backed user record.



Expected output

A valid user is inserted into the users table and the app redirects to the next verification step.

Rejected cases

Empty fields, invalid email, weak password and duplicate email must not corrupt data.

Evidence

Browser screenshot, database row check, one success case and two failure cases.

Git commit

Day 9: implement registration module

Registration lifecycle

The feature is not just one route; it is a full request-response workflow.



Browser responsibility

Collect input and submit name-value pairs.

Flask responsibility

Read form data, validate and decide response.

Database responsibility

Enforce uniqueness and persist valid records.

User experience

Clear error on failure; redirect on success.

```
1 INSERT INTO users (name, email, password_hash)
2 VALUES (%s, %s, %s);
```

GET vs POST for registration

Students must choose the correct HTTP method intentionally.



Method	Used for	Where data appears	Day 9 decision
GET	Display form / search	Query string in URL	Show register.html
POST	Create or modify data	Request body	Create user record

```
1 @app.route('/register', methods=['GET', 'POST'])
2 def register():
3     if request.method == 'GET':
4         return render_template('register.html')
5
6     # POST: read form and create user
```

Rule: GET should not create a new user. POST is used when server state changes.

Registration form anatomy

Every form field must have a useful name.



```
1 <form method="POST" action="{{
url_for('register') }}">
2   <input name="name" required>
3   <input name="email" type="email" required>
4   <input name="password" type="password"
required>
5   <button type="submit">Create Account</button>
6 </form>
```

method="POST"

Submits data in the request body for create/update actions.

action

Tells the browser which route receives the form submission.

name

This is the key Flask uses in request.form.

required / type=email

Helpful client-side checks, but server-side validation is still mandatory.

Reading browser input with request.form

Use `get()` so missing keys do not crash the route.



```
1 name = request.form.get('name', '').strip()
2 email = request.form.get('email',
3 '').strip().lower()
3 password = request.form.get('password', '')
```

.get()

Safely returns a default if the field is missing.

.strip()

Removes extra spaces before validation.

.lower()

Normalizes email before duplicate check.

Untrusted input

All values from the browser must be validated.

Common mistake: `request.form["email"]` throws an error if the field name is wrong or missing.

Server-side validation plan

HTML validation helps UX; Flask validation protects the application.



Input	Validation rule	Failure message
name	Not empty; 2-100 characters	Name is required.
email	Not empty; valid email shape; lowercase	Enter a valid email.
password	Minimum length; policy as decided	Password is too weak.
duplicate	Email must be unique	Email already registered.

```
1 errors = validate_registration(name, email, password)
2 if errors:
3     return render_template('register.html', errors=errors), 400
```

Users table: where registration lands

Registration creates a row in users.



```
1 CREATE TABLE users (  
2   id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,  
3   name VARCHAR(100) NOT NULL,  
4   email VARCHAR(255) NOT NULL UNIQUE,  
5   password_hash VARCHAR(255) NOT NULL,  
6   is_verified BOOLEAN NOT NULL DEFAULT FALSE,  
7   created_at TIMESTAMP NOT NULL DEFAULT  
CURRENT_TIMESTAMP  
8 );
```

id

Primary key: uniquely identifies the user.

email UNIQUE

Database-level protection against duplicates.

password_hash

Store the hash, never the raw password.

is_verified

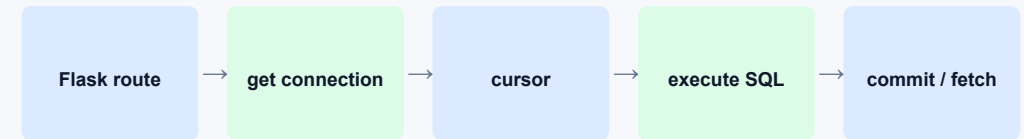
Prepared for Day 10 OTP activation.

Connecting Flask to MySQL

A clean connection helper keeps route code readable.



```
1 import mysql.connector
2
3 def get_db_connection():
4     return mysql.connector.connect(
5         host='localhost',
6         user='root',
7         password='your_password',
8         database='student_notes_manager'
9     )
```



Classroom caution

Do not hardcode production passwords. Later use .env/config.

Debug check

If connection fails, verify server, database name, user and password.

Cursor, execute, commit and fetchone

These four words appear throughout database-backed Flask features.



```
1 connection = get_db_connection()
2 cursor = connection.cursor(dictionary=True)
3
4 cursor.execute('SELECT * FROM users WHERE email=%s',
5 (email,))
6 existing_user = cursor.fetchone()
7
8 cursor.execute(insert_sql, values)
9 connection.commit()
```

cursor

Object used to send SQL to MySQL.

execute()

Runs SQL with separate parameter values.

fetchone()

Reads one result row, useful for duplicate check.

commit()

Permanently saves INSERT/UPDATE/DELETE changes.

Demo idea: comment out commit() and show why the row does not persist.

Parameterized queries

Keep SQL structure separate from user-controlled values.



```
1 -- Good
2 cursor.execute(
3     'SELECT id FROM users WHERE email=%s',
4     (email,)
5 );
6
7 -- Also good
8 cursor.execute(
9     'INSERT INTO users (name,email,password_hash)
VALUES (%s,%s,%s)',
10    (name, email, password_hash)
11 );
```

Do this

Use %s placeholders and pass values as the second argument.

Avoid this

Do not build SQL by concatenating user input strings.

Why

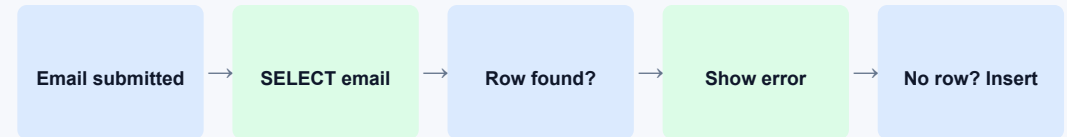
Safer binding prevents SQL injection and quote-handling mistakes.

Duplicate email validation

Application check improves UX; database UNIQUE is the authority.



```
1 cursor.execute(  
2     'SELECT id FROM users WHERE email=%s',  
3     (email,) )  
4 )  
5 existing_user = cursor.fetchone()  
6  
7 if existing_user:  
8     errors.append('Email already registered')
```



Important teaching point

Even after pre-checking, still handle database UNIQUE errors because two requests can happen at the same time.

Student test

Try registering the same email twice and verify the second request fails safely.

Password hashing before insert

Never store the raw password in the database.



```
1 from flask_bcrypt import Bcrypt
2
3 bcrypt = Bcrypt(app)
4
5 password_hash =
bcrypt.generate_password_hash(password).decode()
```



Why hash?

If database leaks, raw passwords should not be visible.

What to store?

Only password_hash goes into users.password_hash.

Day 11 will compare login password against this stored hash.

Reference route: GET + POST



Walk through the route slowly; ask students to name each layer.

```
1 @app.route('/register', methods=['GET', 'POST'])
2 def register():
3     if request.method == 'GET':
4         return render_template('register.html')
5
6     name = request.form.get('name', '').strip()
7     email = request.form.get('email', '').strip().lower()
8     password = request.form.get('password', '')
9
10    errors = validate_registration(name, email, password)
11    if errors:
12        return render_template('register.html', errors=errors), 400
```


Reference route: insert branch

After validation, safely create the user and redirect.



```
1 password_hash =  
  bcrypt.generate_password_hash(password).decode()  
2  
3 cursor.execute(  
4     'INSERT INTO users (name,email,password_hash) VALUES  
(%s,%s,%s)',  
5     (name, email, password_hash)  
6 )  
7 connection.commit()  
8  
9 return redirect(url_for('verify_otp', email=email))
```

Insert only after validation

Do not write partial or invalid data.

commit()

Makes the new user permanent.

redirect()

Prevents duplicate form resubmission and moves user forward.

Next day

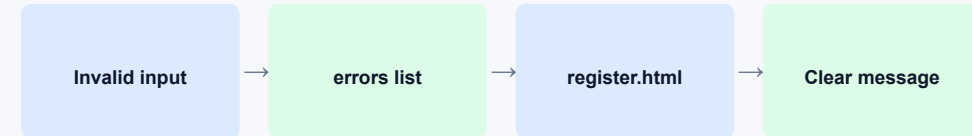
verify_otp route will activate the account.

User feedback: errors and success path

A feature is not complete until users understand what happened.



```
1 {% if errors %}
2   <div class="alert alert-danger">
3     {% for error in errors %}
4       <p>{{ error }}</p>
5     {% endfor %}
6   </div>
7 {% endif %}
```



HTTP 400

Useful when the submitted form is invalid.

Success

Redirect to the next page instead of rendering directly.

Ask students: Should we show "email already exists" or a generic message? Discuss UX vs privacy.

Failure cases students must test

A good registration module proves it handles wrong inputs.



Case	Expected response	Database result
Empty name	Return register page with error	No INSERT
Invalid email	Show validation error	No INSERT
Weak password	Show password error	No INSERT
Duplicate email	Show duplicate error	No second row
DB connection failure	Log error / safe message	No partial commit

Students should capture one successful case and at least two unsuccessful cases as evidence.

Common errors and coaching response

Teach students to identify the failing layer, not guess.



SQL syntax near %s

Parameters were quoted or not passed separately. Fix `execute(sql, values)`.

Unreadable cursor result

Use `dictionary=True` or consistently map tuple positions.

Duplicate despite pre-check

Catch `IntegrityError`; database `UNIQUE` is authoritative.

`request.form` returns empty

Check input name attributes and form method.

No row saved

Check `connection.commit()`, database name and table schema.

Template error

Verify route passes errors and templates folder names match.

Trainer live-coding plan

Build the feature in thin, testable slices.



Step 1

Open Day 8 project and confirm `/register` renders.

Step 2

Add POST method and print `request.form` for inspection.

Step 3

Add validation and show errors in template.

Step 4

Connect MySQL and test SELECT duplicate check.

Step 5

Hash password and INSERT valid user.

Step 6

Verify row in MySQL and commit to Git.

Guided lab: implement Day 9 milestone

Students reproduce the trainer version, then add one independent improvement.



Required feature

Register page submits name, email and password using POST.

Required validation

Missing, invalid and duplicate inputs must be rejected.

Required database work

Use parameterized INSERT and commit only after successful validation.

Independent improvement

Better validation, reusable helper, accessibility improvement or extra test case.

Evidence

Screenshots + MySQL row check + failure case evidence.

Commit

Day 9: implement registration module

Day 9 quick quiz

Use this as oral check or exit ticket.



Expected answer style

Students must answer using flow language: browser -> Flask -> validation -> MySQL -> response.

Trainer check

Ask one student to draw the registration flow without looking at notes.

1. Why is POST used for registration?
2. What does `request.form.get("email")` return?
3. Why should we call `strip()` and `lower()` on email?
4. Why is HTML required not enough validation?
5. What does `cursor.execute()` do?
6. What does `commit()` do?
7. Why should SQL values be parameterized?
8. Why is email UNIQUE still needed after duplicate check?
9. Why do we store `password_hash`, not `password`?
10. What should happen after successful registration?

Homework assignment & recap

Submit a working Day-9 registration milestone before the next class.



Task 1

Implement /register GET and POST.

Task 2

Validate name, email and password.

Task 3

Check duplicate email using MySQL.

Task 4

Hash password and insert user.

Task 5

Test one success and two failures.

Task 6

Update README and commit cleanly.

Recap: Registration form -> request.form -> validation -> duplicate check -> bcrypt hash -> parameterized INSERT -> commit -> redirect.

Next class: Email communication and OTP verification.